Задания. Раздел 1. Отчет в файле task3_part1.pdf

1. Войдите под пользователем user1 из практики 2 (su - user1)
2. Подсчитайте количество процессов, имеющих несколько потоков выполнения
3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:
   - удалите поля VIRT, RES, SHR;
   - добавьте поле RUSER и сделайте так, чтобы это поле было показано после поля USER;
4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля
5. Перейдите в терминальное окно с top и выполните следующие действия:
   - выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;
   - найдите процесс, запущенный командой passwd;
   - отправьте этому процессу сигналы 15 (SIGTERM), 2 (SIGINT), 3 (SIGQUIT), 9(SIGKILL)

1.
```
root@eltex-practice2-pg2-v9:~# sudo su - user1
You are required to change your password immediately (password expired).
Changing password for user1.
Current password:
New password:
Retype new password:
The password has not been changed.
New password:
Retype new password:
user1@eltex-practice2-pg2-v9:~$
```

2.
```
user1@eltex-practice2-pg2-v9:~$ ps -e -o nlwp,pid,cmd | awk '$1 > 1'
wc -l
11
```

3.
```
user1@eltex-practice2-pg2-v9:~$ top
```

[f]

```
Fields Management for window 1:Def, whose current sort field is %CPU
   Navigate with Up/Dn, Right selects for move then <Enter> or Left com
   'd' or <Space> toggles display, 's' sets sort.  Use 'q' or <Esc> to

* PID      = Process Id            nsIPC    = IPC namespace Inode
* USER     = Effective User Name   nsMNT    = MNT namespace Inode
* PR       = Priority              nsNET    = NET namespace Inode
* NI       = Nice Value            nsPID    = PID namespace Inode
* VIRT     = Virtual Image (KiB)   nsUSER   = USER namespace Inode
* RES      = Resident Size (KiB)   nsUTS    = UTS namespace Inode
* SHR      = Shared Memory (KiB)   LXC      = LXC container name
* S        = Process Status        RSan     = RES Anonymous (KiB)
* %CPU     = CPU Usage             RSfd     = RES File-based (KiB)
* %MEM     = Memory Usage (RES)    RSlk     = RES Locked (KiB)
* TIME+    = CPU Time, hundredths  RSsh     = RES Shared (KiB)
* COMMAND  = Command Name/Line     CGNAME   = Control Group name
  PPID     = Parent Process pid    NU       = Last Used NUMA node
  UID      = Effective User Id     LOGID    = Login User Id
  RUID     = Real User Id          EXE      = Executable Path
  RUSER    = Real User Name        RSS      = Res Mem (smaps), KiB
  SUID     = Saved User Id         PSS      = Proportion RSS,  KiB
  SUSER    = Saved User Name       PSan     = Proportion Anon, KiB
  GID      = Group Id              PSfd     = Proportion File, KiB
  GROUP    = Group Name            PSsh     = Proportion Shrd, KiB
  PGRP     = Process Group Id      USS      = Unique RSS, KiB
  TTY      = Controlling Tty       ioR      = I/O Bytes Read
  TPGID    = Tty Process Grp Id    ioRop    = I/O Read Operations
  SID      = Session Id            ioW      = I/O Bytes Written
  nTH      = Number of Threads     ioWop    = I/O Write Operations
  P        = Last Used Cpu (SMP)   AGID     = Autogroup Identifier
  TIME     = CPU Time              AGNI     = Autogroup Nice Value
  SWAP     = Swapped Size (KiB)    STARTED  = Start Time from boot
  CODE     = Code Size (KiB)       ELAPSED  = Elapsed Running Time
  DATA     = Data+Stack (KiB)      %CUU     = CPU Utilization
  nMaj     = Major Page Faults     %CUC     = Utilization + child
  nMin     = Minor Page Faults     nsCGROUP = CGRP namespace Inode
  nDRT     = Dirty Pages Count     nsTIME   = TIME namespace Inode
  WCHAN    = Sleeping in Function
  Flags    = Task Flags <sched.h>
  CGROUPS  = Control Groups
  SUPGIDS  = Supp Groups IDs
  SUPGRPS  = Supp Groups Names
  TGID     = Thread Group Id
  OOMa     = OOMEM Adjustment
  OOMs     = OOMEM Score current
  ENVIRON  = Environment vars
  vMj      = Major Faults delta
  vMn      = Minor Faults delta
  USED     = Res+Swap Size (KiB)
```

```
Fields Management for window 1:Def, whose current sort field is %CPU
   Navigate with Up/Dn, Right selects for move then <Enter> or Left com
   'd' or <Space> toggles display, 's' sets sort.  Use 'q' or <Esc> to

* PID      = Process Id            nsIPC    = IPC namespace Inode
* USER     = Effective User Name   nsMNT    = MNT namespace Inode
* RUSER    = Real User Name        nsNET    = NET namespace Inode
* PR       = Priority              nsPID    = PID namespace Inode
* NI       = Nice Value            nsUSER   = USER namespace Inode
  VIRT     = Virtual Image (KiB)   nsUTS    = UTS namespace Inode
  RES      = Resident Size (KiB)   LXC      = LXC container name
  SHR      = Shared Memory (KiB)   RSan     = RES Anonymous (KiB)
* S        = Process Status        RSfd     = RES File-based (KiB)
* %CPU     = CPU Usage             RSlk     = RES Locked (KiB)
* %MEM     = Memory Usage (RES)    RSsh     = RES Shared (KiB)
* TIME+    = CPU Time, hundredths  CGNAME    = Control Group name
* COMMAND  = Command Name/Line     NU       = Last Used NUMA node
  PPID     = Parent Process pid    LOGID    = Login User Id
  UID      = Effective User Id     EXE      = Executable Path
  RUID     = Real User Id          RSS      = Res Mem (smaps), KiB
```

```
top - 06:03:24 up 4 days,  8:23,  1 user,  load average: 0.00, 0.00, 0.
Tasks: 113 total,   1 running, 112 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 98.5 id,  1.5 wa,  0.0 hi,  0.0 si,
MiB Mem :   3915.9 total,   2294.1 free,    501.7 used,   1391.8 buff/c
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3414.2 avail

    PID USER      RUSER     PR  NI S  %CPU  %MEM     TIME+ COMMAND
    666 root      root      20   0 S   0.3   0.1   3:03.23 qemu-ga
      1 root      root      20   0 S   0.0   0.3   0:05.12 systemd
      2 root      root      20   0 S   0.0   0.0   0:00.03 kthreadd
      3 root      root      20   0 S   0.0   0.0   0:00.00 pool_workqu+
      4 root      root       0 -20 I   0.0   0.0   0:00.00 kworker/R-r+
```

4.

```
user1@eltex-practice2-pg2-v9:~$ passwd
Changing password for user1.
Current password:
```

```
top - 06:06:26 up 4 days,  8:26,  1 user,  load average: 0.00, 0.00, 0
Tasks: 117 total,   1 running, 116 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si
MiB Mem :   3915.9 total,   2293.6 free,    501.5 used,   1392.4 buff/
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3414.3 avail
Which user (blank for all) user1
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+
    666 root      20   0   80712   4316   4016 S   0.3   0.1   3:03.32
      1 root      20   0   22112  13360   9556 S   0.0   0.3   0:05.12
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.03
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00
```

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
17739 user1     20   0    8644   5596   3888 S   0.0   0.1   0:00.01 bash
17763 user1     20   0    8292   4092   3784 S   0.0   0.1   0:00.00 screen
17764 user1     20   0    8952   3320   2132 S   0.0   0.1   0:00.04 screen
17765 user1     20   0    8668   5556   3912 S   0.0   0.1   0:00.00 bash
17773 user1     20   0    8668   5540   3892 S   0.0   0.1   0:00.00 bash
17781 user1     20   0    8668   5568   3920 S   0.0   0.1   0:00.00 bash
17796 user1     20   0   11948   6104   3868 R   0.0   0.2   0:00.11 top
```

5. У меня не отображается passwd , я не выяснил почему. Решил попробовать с nano, nano отображается корректно.

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
17739 user1     20   0    8644   5596   3888 S   0.0   0.1   0:00.01 bash
17763 user1     20   0    8292   4092   3784 S   0.0   0.1   0:00.00 screen
17764 user1     20   0    8952   3320   2132 S   0.0   0.1   0:00.04 screen
17765 user1     20   0    8668   5556   3912 S   0.0   0.1   0:00.00 bash
17773 user1     20   0    8668   5540   3892 S   0.0   0.1   0:00.00 bash
17781 user1     20   0    8668   5568   3920 S   0.0   0.1   0:00.00 bash
17796 user1     20   0   11948   6104   3868 R   0.0   0.2   0:00.14 top
17813 user1     20   0    7188   3988   2948 S   0.0   0.1   0:00.00 nano
```

```
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.
PID to signal/kill [default pid = 17739] 17813
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %M
17739 user1     20   0    8644   5596   3888 S   0.0   0
```

```
Send pid 17813 signal [15/sigterm] 15
  PID USER      PR  NI    VIRT    RES
```

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
17739 user1     20   0    8644   5596   3888 S   0.0   0.1   0:00.01 bash
17763 user1     20   0    8292   4092   3784 S   0.0   0.1   0:00.00 screen
17764 user1     20   0    8952   3320   2132 S   0.0   0.1   0:00.04 screen
17765 user1     20   0    8668   5556   3912 S   0.0   0.1   0:00.00 bash
17773 user1     20   0    8668   5540   3892 S   0.0   0.1   0:00.00 bash
17781 user1     20   0    8668   5568   3920 S   0.0   0.1   0:00.00 bash
17796 user1     20   0   11948   6104   3868 R   0.0   0.2   0:00.19 top
```

nano пропал из top

```
^G Help        ^O Write Out   ^W Where I
Received SIGHUP or SIGTERMe   ^\ Replace
user1@eltex-practice2-pg2-v9:~$
```

```
Send pid 17814 signal [15/sigterm] 2
  PID USER      PR  NI    VIRT    RES
```

```
Send pid 17814 signal [15/sigterm] 3
  PID USER      PR  NI    VIRT    RES
```

После 9

```
  GNU nano 7.2
Killed
user1@eltex-practice2-pg2-v9:~$
```

Задания. Раздел 1.

6. Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z
7. Выполните команду sleep 600, нажмите Ctrl-Z и выполните команду jobs
8. Последнее задание (sleep 600) сделайте фоновым
9. Измените число NICE у задания (sleep 600), сделав его равным 10
10. Проверьте, что число NICE у этого задания изменилось
11. Сделайте задание vim ~/file_ task3.txt активным и выйдите из редактора
12. Отправьте сигнал 15 (SIGTERM) заданию sleep 600 и выполните команду jobs
13. Создайте перехватчик сигналов SIGINT и SIGQUIT внутри командного интерпретатора, который выводит сообщение «Меня голыми руками не возьмёшь!» (используйте встроенную команду trap) и отправьте сигналы самому себе

6.

```
user1@eltex-practice2-pg2-v9:~$ vim ~/.file_task3.txt
```

```
~
"~/.file_task3.txt" [New]

[1]+  Stopped                 vim ~/.file_task3.txt
user1@eltex-practice2-pg2-v9:~$
```

7.

```
user1@eltex-practice2-pg2-v9:~$ sleep 600
^Z
[2]+  Stopped                 sleep 600
```

```
user1@eltex-practice2-pg2-v9:~$ jobs
[1]-  Stopped                 vim ~/.file_task3.txt
[2]+  Stopped                 sleep 600
```

8.

```
user1@eltex-practice2-pg2-v9:~$ bg %2
[2]+ sleep 600 &
user1@eltex-practice2-pg2-v9:~$ jobs
[1]+  Stopped                 vim ~/.file_task3.txt
[2]-  Running                 sleep 600 &
```

9.

```
user1@eltex-practice2-pg2-v9:~$ renice 10 %2
renice: bad process ID value: %2
user1@eltex-practice2-pg2-v9:~$ ps aux | grep sleep
user1      17845  0.0  0.0   5684  2168 pts/3    S    06:25   0:00 sleep 600
user1      17849  0.0  0.0   6544  2396 pts/3    S+   06:28   0:00 grep --color=auto slee
user1@eltex-practice2-pg2-v9:~$ renice 10 17845
17845 (process ID) old priority 0, new priority 10
```

10.
```
top - 06:29:26 up 4 days,  8:49,  1 user,  load average: 0.00, 0.01, 0.00
Tasks:   1 total,   0 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.2 us,  0.0 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2248.0 free,    517.3 used,   1422.5 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3398.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  17845 user1     30  10    5684   2168   2056 S   0.0   0.1   0:00.00 sleep
```

11.
```
user1@eltex-practice2-pg2-v9:~$ fg %1
```

12.
```
~
".file_task3.txt" [New] 0L, 0B written
user1@eltex-practice2-pg2-v9:~$ jobs
[2]+  Running                 sleep 600 &
```

```
user1@eltex-practice2-pg2-v9:~$ kill -15 %2
user1@eltex-practice2-pg2-v9:~$ jobs
[2]+  Terminated              sleep 600
```

13.
```
user1@eltex-practice2-pg2-v9:~$ trap 'echo "Меня голыми руками не возьмешь!"' SIGINT SIGQU
IT
```

```
user1@eltex-practice2-pg2-v9:~$ echo $$
17773
```

во втором окне

```
user1@eltex-practice2-pg2-v9:~$ kill -2 17773
```

в первом окне

```
user1@eltex-practice2-pg2-v9:~$ echo $$
17773
Меня голыми руками не возьмешь! ^C

user1@eltex-practice2-pg2-v9:~$
```

Задания. Раздел 2.

1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:
   - Создает именованный канал с именем cuckoo.pid в каталоге /tmp/run/
   - Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!
   - В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?" возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N
   - При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу.
2. Запустите скрипт cuckoo.sh при помощи nohup или screen

1.
```
#!/bin/bash

LOG_FILE="cuckoo.log"
PIPE="/tmp/run/cuckoo.pid"

mkdir -p /tmp/run

rm -f "$PIPE"
mkfifo "$PIPE"

echo "$(date '+%Y-%m-%d %H:%M:%S') Startup!" >> "$LOG_FILE"

trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> "$LOG_FILE"; rm -f "$PIPE"; exit 0' SIGINT

while read -r request; do
        if [[ "$request" =~ ^([^:]+):.*how\ much\ time.*$ ]]; then
                part="${BASH_REMATCH[1]}"
                if [[ "$part" =~ ^([^[]+)\[([0-9]+)\]$ ]]; then
                NAME="${BASH_REMATCH[1]}"
                PID="${BASH_REMATCH[2]}"
                N=$(( RANDOM % 9 + 2))
                echo "$(date '+%Y-%m-%d %H:%M:%S') ${NAME}[${PID}] ${N}" >> "$LOG_FILE"
                fi
        fi
done < "$PIPE"
```

2.
```
user1@eltex-practice2-pg2-v9:~/cuckoo$ chmod +x cuckoo.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ ./cuckoo.sh
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ echo "myprocess[1234]: how much time do I have?" > /tmp/run/cuckoo.pid
user1@eltex-practice2-pg2-v9:~/cuckoo$ cat /tmp/run/cuckoo.pid
^C
user1@eltex-practice2-pg2-v9:~/cuckoo$ cat *.log
2026-04-08 07:09:00 Startup!
2026-04-08 07:17:56 Startup!
2026-04-08 07:18:13 myprocess[1234] 6
```

Задания. Раздел 2.

3. Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:
   - При запуске проверят, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»
   - При запуске дописывает в файл report_**имя_скрипта_без_полного_пути**.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен
   - Запрашивает через именованный канал скрипта cuckoo время сообщением в указанном ранее формате и ждет такое количество секунд.
   - Дописывает в файл report_**имя_скрипта_без_полного_пути**.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.
4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log

3.

```bash
#!/bin/bash

SCRIPT_NAME=$(basename "$0")
if [[ "$SCRIPT_NAME" == "template_task.sh" ]]; then
    echo "я бригадир, сам не работаю"
    exit 1
fi
LOG_FILE="report_${SCRIPT_NAME}.log"
CUCKOO_LOG="cuckoo.log"
PIPE="/tmp/run/cuckoo.pid"

log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] $1" >> "$LOG_FILE"
}

log_message "Скрипт запущен"
if [[ ! -p "$PIPE" ]]; then
    log_message "Ошибка: именованный канал $PIPE не найден"
    echo "Ошибка: cuckoo.sh не запущен?" >&2
    exit 1
fi

REQUEST="${SCRIPT_NAME}[$$]: how much time do I have?"
echo "$REQUEST" > "$PIPE"
sleep 0.5

CUCKOO_RESPONSE=$(grep "\[$$\]" "$CUCKOO_LOG" | tail -1)

if [[ -z "$CUCKOO_RESPONSE" ]]; then
    log_message "Ошибка: не найден ответ от cuckoo для PID $$"
    echo "Ошибка: cuckoo не ответил" >&2
    exit 1
fi
N=$(echo "$CUCKOO_RESPONSE" | awk '{print $NF}')
if [[ ! "$N" =~ ^[0-9]+$ ]] || [[ "$N" -lt 2 ]] || [[ "$N" -gt 10 ]]; then
    log_message "Ошибка: получен некорректный ответ от cuckoo: '$N'"
    echo "Ошибка: cuckoo вернул некорректное значение" >&2
    exit 1
fi
log_message "Получено время ожидания: $N секунд"
sleep "$N"
log_message "Скрипт завершился, работал $N секунд"
echo "Скрипт выполнен. Ожидал $N секунд."
exit 0
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ chmod +x template_task.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ cp template_task.sh worker1.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ ./worker1.sh
Скрипт выполнен. Ожидал 10 секунд.
```

4.

```bash
#!/bin/bash

CONFIG_FILE="observer.conf"
LOG_FILE="observer.log"

log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] $1" >> "$LOG_FILE"
}

if [[ ! -f "$CONFIG_FILE" ]]; then
    log_message "Ошибка: файл конфигурации $CONFIG_FILE не найден"
    echo "Ошибка: observer.conf не существует"
    exit 1
fi

log_message "Наблюдатель запущен"

while IFS= read -r script_name; do
    [[ -z "$script_name" ]] && continue
    [[ "$script_name" =~ ^#.*$ ]] && continue
    script_name=$(echo "$script_name" | xargs)

    found=0
    for pid in /proc/[0-9]*; do
        if [[ -f "$pid/comm" ]]; then
            proc_name=$(cat "$pid/comm" 2>/dev/null)
            if [[ "$proc_name" == "$script_name" ]]; then
                found=1
                break
            fi
        fi
    done

    if [[ $found -eq 0 ]]; then
        if [[ -f "./$script_name" ]]; then
            nohup ./"$script_name" > /dev/null 2>&1 &
            log_message "Перезапущен скрипт: $script_name (PID: $!)"
        else
            log_message "Ошибка: скрипт $script_name не найден в текущем каталоге"
        fi
    fi
done < "$CONFIG_FILE"

log_message "Наблюдатель завершил проверку"
```

observer.conf:

```
cuckoo.sh
worker1.sh
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ ./observer.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ cat observer.log
2026-04-08 07:40:57 [18195] Наблюдатель запущен
2026-04-08 07:40:58 [18195] Перезапущен скрипт: cuckoo.sh (PID: 18437)
2026-04-08 07:40:58 [18195] Перезапущен скрипт: worker1.sh (PID: 18685)
2026-04-08 07:40:58 [18195] Наблюдатель завершил проверку
```

Задания. Раздел 2.

5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту
6. Создайте несколько символьных ссылок на файл template_task.sh с различными именами (рабочие задачи), добавьте в файл конфигурации observer.conf соответствующие записи об этих задачах, включая исходный файл template_task.sh
7. Соберите статистику работы в виде набора файлов report_*.log, observer.log, cuckoo.log приложите их вместе с исходными текстами скриптов в качестве отчета в виде сжатого архива tar. Не забудьте остановить процесс, удалив задачу в cron!

5.

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ crontab -e
```

```
  GNU nano 7.2                    /tmp/crontab.oCnXo7/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /home/user1/cuckoo/observer.sh
```

6.

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ ln -s template_task.sh task1.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ ln -s template_task.sh task2.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ ln -s template_task.sh task3.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ ln -s template_task.sh worker1.sh
ln: failed to create symbolic link 'worker1.sh': File exists
user1@eltex-practice2-pg2-v9:~/cuckoo$ ln -s template_task.sh worker2.sh
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ cat observer.conf
cuckoo.sh
worker1.sh
template_task.sh
task1.sh
task2.sh
task3.sh
worker2.sh
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ ls
cuckoo.log  observer.conf  observer.sh            task1.sh  task3.sh          worker1.sh
cuckoo.sh   observer.log   report_worker1.sh.log  task2.sh  template_task.sh  worker2.sh
```

```
user1@eltex-practice2-pg2-v9:~/cuckoo$ tar -czf completed_task.tar.gz *
user1@eltex-practice2-pg2-v9:~/cuckoo$ ls
completed_task.tar.gz  observer.conf  report_worker1.sh.log  task3.sh          worker2.sh
cuckoo.log             observer.log   task1.sh               template_task.sh
cuckoo.sh              observer.sh    task2.sh               worker1.sh
user1@eltex-practice2-pg2-v9:~/cuckoo$ tar -tzf completed_task.tar.gz
cuckoo.log
cuckoo.sh
observer.conf
observer.log
observer.sh
report_worker1.sh.log
task1.sh
task2.sh
task3.sh
template_task.sh
worker1.sh
worker2.sh
```

7. Задачу в кронтабе удалил.